AGENTIC AI

WEEK 1

Foundation & Environment Setup | Agentic Architecture | LLM Orchestration | AI Frameworks

Document Type	Personal Study Reference — Publication Draft
Domain	Generative AI / Agentic Systems / LLM Engineering
Week	Week 1 of 8 — Foundations
Topics Covered	Environment Setup, Agents, Workflows, Orchestration, Frameworks (MCP, CrewAI, LangGraph, AutoGen)

This document is part of an ongoing personal learning journal targeting production-ready GenAI and Data Science applications. Each chapter is structured with a formal theory section followed by annotated practical code.

TABLE OF CONTENTS

SECTION 1

Theoretical Foundations

This section provides a rigorous theoretical grounding in Agentic AI — from the initial development environment setup through to the conceptual architecture of autonomous multi-agent systems and the major orchestration frameworks employed in production-grade applications.

1.1 Development Environment Setup

Tools and configurations required for building Agentic AI applications

1.1.1 Visual Studio Code (VS Code)

Visual Studio Code (VS Code) serves as the primary Integrated Development Environment (IDE) for this course. Developed and maintained by Microsoft, VS Code is a lightweight yet highly extensible editor that supports virtually every programming language through its plugin ecosystem. For Python-based Agentic AI development, VS Code provides several indispensable capabilities: intelligent code completion via IntelliSense, integrated terminal access, a native debugging interface, Git source control integration, and seamless support for Jupyter notebooks directly within the editor.

Its extensibility is a critical advantage — extensions such as the Python extension (by Microsoft), Pylance (for static type checking), and GitHub Copilot (for AI-assisted development) transform VS Code into a full-featured Python development environment without the overhead of a heavier IDE such as PyCharm.

1.1.2 Python Virtual Environments

A Python virtual environment is an isolated directory that contains its own Python interpreter and a self-contained collection of installed packages. Virtual environments are foundational to professional Python development because they solve the dependency conflict problem: different projects may require different (and potentially incompatible) versions of the same library. Without isolation, installing a package for one project may break another.

In this course, the standard library module `venv` is used to create lightweight, self-contained environments. Each Agentic AI project should be developed inside its own dedicated virtual environment. This practice ensures reproducibility, clean deployments, and straightforward dependency management.

Why Virtual Environments Matter

Consider a scenario where Project A requires openai==0.28 and Project B requires openai==1.x. Without virtual environments, both cannot coexist on the same machine. Separate environments provide each project its own dependency namespace, entirely avoiding this conflict.

1.1.3 Git & Version Control

Git is a distributed version control system that tracks changes in source code during software development. For AI projects — where experiments are iterative, models are frequently adjusted, and code evolves rapidly — Git is indispensable. It allows practitioners to maintain a complete history of all changes, revert to any prior state, create experimental branches without affecting the main codebase, and collaborate with others efficiently.

Cursor is an AI-enhanced code editor built on top of VS Code, incorporating an integrated LLM assistant that can answer code questions, explain functions, and even generate complete implementations. It is particularly valuable in an Agentic AI learning context as it provides contextual AI assistance inline within the development workflow.

1.2 Secure API Key Management

Using dotenv for environment variable isolation

When developing applications that interact with external AI services — such as OpenAI, Anthropic, or Google — API keys serve as authentication credentials. These keys are sensitive secrets: if exposed publicly (e.g., committed to a GitHub repository), they can be exploited by malicious actors, leading to unauthorized usage and significant financial charges.

The industry-standard approach to managing API keys in Python projects is the `python-dotenv` package, which loads key-value pairs from a `.env` file into the process's environment variables at runtime. This separates secrets from source code entirely.

How it Works

- A `.env` file is created at the root of the project directory and populated with API keys in the format `KEY_NAME=value`.
- The `.env` file is added to `.gitignore` to ensure it is never committed to version control.
- At runtime, `load_dotenv()` reads the `.env` file and injects its variables into the operating system's environment namespace.
- The `os.getenv('KEY_NAME')` function then retrieves these values safely within Python code.
- The `override=True` parameter ensures that if the variable already exists in the OS environment, the value in `.env` takes precedence — useful in environments with conflicting configurations.

Security Best Practice

Never hardcode API keys directly in Python scripts. Always use environment variables loaded via dotenv. Additionally, add your .env file to .gitignore immediately upon project creation, before making any commits.

1.3 LLM Model Integration & the OpenAI-Compatible API

Understanding the chat completions interface and multi-provider support

The OpenAI Python client library establishes a de facto standard for interacting with Large Language Models via API. Crucially, many alternative providers — including Google (Gemini) and Groq — have adopted OpenAI-compatible API interfaces. This means that with a single, unified client, one can switch between different underlying models by simply changing the `api_key` and `base_url` parameters, without altering any application logic.

The Message Format

All OpenAI-compatible APIs communicate through a structured message format — a list of Python dictionaries, each representing a single conversational turn. Every dictionary contains two mandatory keys:

- **role**: Defines the speaker. Valid values are `"system"` (sets the AI's persona/instructions), `"user"` (the human's message), and `"assistant"` (a prior AI response included for multi-turn context).
- **content**: The actual textual content of that conversational turn.

This list-based structure allows the API to maintain full conversational context. By passing the entire conversation history with each request, the model can produce coherent, contextually aware responses across multiple turns — a design that is stateless at the API level but stateful at the application level.

The Chat Completions Endpoint

The primary method for generating responses is `client.chat.completions.create()`. This method accepts the model identifier and the messages list as its core parameters, dispatches the request to the provider's inference infrastructure, and returns a structured response object. The generated text is accessed via `response.choices[0].message.content` — the `choices` list accommodates scenarios where the API is configured to return multiple candidate responses.

1.4 Agents & Agentic Architecture

Defining the foundational building blocks of autonomous AI systems

An Agent, in the context of artificial intelligence, is a computational system in which the output produced by a Large Language Model determines — dynamically and autonomously — the sequence of subsequent tasks to be executed. This is the foundational distinction between a simple LLM query-response pattern and a true agentic system: in an agent, the model does not merely respond; it decides, plans, and acts.

The term *Agentic AI* is applied to any system that exhibits one or more of the following defining hallmarks:

Hallmark	Description
Multiple LLM Calls	The system involves multiple sequential or conditional invocations of a language model, where each call may build upon, refine, or redirect the output of prior calls.
Tool Usage	The LLM is granted the ability to invoke external functions, APIs, or services — such as web search engines, code interpreters, databases, or file systems — to gather information or take actions beyond text generation.
Interactive Environment	Multiple LLM instances or agents operate within a shared environment, interacting with each other in a coordinated, goal-directed manner.
Planner & Autonomy	A dedicated planning component — often itself an LLM — decomposes complex, high-level goals into actionable sub-tasks and assigns them to appropriate agents, with minimal human intervention.

Key Insight

It is important to understand that an 'agent' is not necessarily a single model. An agentic system is better understood as an architecture — a coordinated arrangement of models, tools, planners, and execution environments working together to achieve goals that would be beyond the capability of any single, isolated LLM call.

1.5 Agentic Systems: Workflows vs. Agents

Understanding the critical distinction between deterministic and autonomous systems

A critical conceptual distinction in Agentic AI is that between *Workflows* and *Agents*. While both involve LLMs, they differ fundamentally in the degree of autonomy afforded to the model and in the determinism of execution paths.

Workflows

A Workflow is a system in which LLMs and external tools are orchestrated through **predefined, deterministic code paths** authored by the developer. The flow of execution is fixed at design time. The LLM contributes intelligence at specific, designated nodes within a pre-constructed pipeline, but it does not alter the pipeline itself. Workflows are predictable, auditable, and well-suited to tasks with clearly defined structure.

Agents

An Agent is a system in which an LLM **dynamically directs its own processes and tool usage**. The model itself decides, at runtime, which tools to invoke, in what order, and whether additional LLM calls are required. The execution path is not fixed by the developer — it emerges from the model's reasoning about the task at hand. Agents are open-ended, adaptive, and capable of handling novel situations, but they are inherently less predictable than workflows.

Dimension	Workflows	Agents
Execution Path	Predefined, deterministic	Dynamic, emergent at runtime
Autonomy Level	Low — developer-directed	High — model-directed
Predictability	High — fully auditable	Lower — path is non-deterministic
Flexibility	Lower — handles known scenarios	Higher — handles novel scenarios
Use Case	Structured, repetitive processes	Complex, open-ended tasks

1.6 Workflow Design Patterns

Prompt Chaining, Routing, and Parallelization

Even within the constrained, predefined structure of workflows, several powerful design patterns have emerged that significantly expand the capability of LLM-driven systems. These patterns are the foundational building blocks from which more sophisticated agentic architectures are constructed.

1.6.1 Prompt Chaining

Prompt Chaining involves constructing a **sequence of LLM calls** in which the output of one call becomes the input to the next. This pattern is employed when a task is too complex or too nuanced to be reliably completed in a single LLM invocation. By decomposing the task into a series of smaller, well-defined steps — each handled by a focused LLM call — the overall quality and reliability of the output is substantially improved.

A practical example is a multi-step content pipeline: the first LLM call generates a raw draft; the second call critically reviews and annotates the draft; the third call rewrites the draft incorporating the review feedback. A programmatic *gate* may exist between calls to check whether the output of a step meets a defined quality threshold before passing it forward — routing to an 'Exit' path if it fails.

1.6.2 Routing

Routing is the practice of **directing an incoming input to a specialized sub-task or sub-agent** based on the nature of that input. A classifier LLM (or a rule-based system) first categorizes the input, and the result of this classification determines which specialized downstream process handles it. This pattern improves operational clarity and allows each sub-system to be optimized for its specific domain.

For instance, a customer support system might route technical queries to a technical support agent, billing inquiries to a billing agent, and general questions to a general-purpose agent — all from a single entry point.

1.6.3 Parallelization

Parallelization decomposes a complex task into **multiple independent sub-tasks that are executed concurrently** by separate agents or LLM calls. Upon completion, an *Aggregator* component collects and synthesizes the outputs of all parallel executions into a final, coherent result. This pattern is well-suited to tasks that can be partitioned without interdependencies, and it dramatically reduces end-to-end latency compared to sequential execution.

1.7 Advanced Workflow Patterns

Orchestrator-Worker and the Critic-Optimizer loop

1.7.1 Orchestrator-Worker Pattern

In the Orchestrator-Worker pattern, a high-level **Orchestrator LLM** is responsible for receiving a complex task, dynamically decomposing it into component sub-tasks, and delegating each sub-task to a specialized **Worker LLM or agent**. The Orchestrator also manages the reassembly of individual worker outputs into a complete, coherent final result.

This pattern closely mirrors organizational management structures and is particularly powerful for tasks that are too complex or multi-faceted for a single model to handle reliably. The Orchestrator

maintains situational awareness of the overall goal while individual Workers focus narrowly on their assigned sub-problems.

1.7.2 Critic-Optimizer Pattern

The Critic-Optimizer pattern introduces a **quality assurance feedback loop** between two LLMs. A primary *Generator* LLM produces an initial output. A secondary *Critic* LLM then independently evaluates this output against defined quality criteria, and produces a verdict: either accepting the output as satisfactory, or rejecting it with specific, actionable reasons for rejection. If rejected, these reasons are fed back to the Generator, which refines its output accordingly. This loop continues until the Critic's evaluation threshold is met.

This pattern is considered highly effective in production systems because it replicates the principle of peer review: independent evaluation catches errors and weaknesses that the original generator may be blind to. It is particularly valuable for tasks requiring high accuracy, such as code generation, legal document drafting, or medical text summarization.

Why Critic-Optimizer is Powerful

Research in LLM alignment has demonstrated that LLMs are substantially better at evaluating and critiquing text than they are at generating perfect text from scratch on the first attempt. The Critic-Optimizer pattern leverages this asymmetry to systematically elevate output quality through iterative refinement.

1.8 The Agentic Feedback Loop

How autonomous agents perceive, act, and learn from their environment

The Agentic Feedback Loop is the operational mechanism through which autonomous AI agents function. It describes the cyclical process of perception, reasoning, action, observation, and adaptation that an agent executes in pursuit of a defined goal. Understanding this loop is essential to understanding how agentic systems differ from simple request-response LLM interactions.

1. Human Initiation

A human user provides a high-level goal or task description. This initiates the agentic loop.

2. LLM Reasoning & Action Planning

The LLM processes the request, reasons about the goal, and generates an 'Action' — a concrete step to be taken. This may involve calling a tool, querying a database, generating code, or producing text.

3. Action Execution

The planned action is executed within the environment. External tools such as image generation models (e.g., Stable Diffusion), speech transcription models (e.g., Whisper), code interpreters, or web search APIs may be invoked.

4. Environment Feedback / Observation

The results of the executed action — the tool's output, an error message, a retrieved document, or any other environmental response — are fed back to the LLM as new context.

5. Learning & Adaptation

The LLM processes this feedback, updates its understanding of the task's state, and generates a new Action. It adapts its strategy based on what it has observed.

6. Goal Achievement / Termination

Steps 3–5 repeat until either the goal is achieved and the agent terminates, a maximum iteration limit is reached, or the agent determines it cannot complete the task.

Analogy: The Scientific Method

The agentic feedback loop closely mirrors the scientific method: form a hypothesis (action plan), conduct an experiment (execute action), observe results (environment feedback), revise the hypothesis (adaptation), and repeat. This iterative refinement toward a goal is what makes agentic systems qualitatively more powerful than single-shot LLM queries.

1.9 Drawbacks & Guardrails

Limitations, risks, and control mechanisms in agentic systems

Despite their considerable capabilities, agentic systems introduce unique challenges and risks that must be addressed through careful design. Awareness of these limitations is a prerequisite for building responsible, production-ready agentic applications.

Unpredictable Execution Paths: Unlike deterministic workflows, agents select their own action sequences at runtime. The path taken to achieve a goal may be entirely unexpected, making it difficult to reason about system behaviour in advance, to test exhaustively, or to audit after the fact.

Unpredictable Time & Cost: The number of LLM calls, tool invocations, and iterations required to complete a task is determined dynamically. In the worst case, an agent may enter extended loops or explore highly inefficient solution paths, leading to unpredictable latency and API cost accumulation.

Monitoring & Visibility Challenges: When an agent is executing a multi-step plan, it may not be immediately apparent — even to the developers — precisely what the agent is doing, why it chose a particular action, or what state it currently holds. This opacity is a significant operational challenge.

Error Propagation: A mistake made early in an agentic loop can compound through subsequent iterations. If the LLM's initial reasoning is flawed, subsequent actions may build upon that flaw,

leading the agent further from the correct solution rather than toward it.

Guardrails

Guardrails are constraints, validation mechanisms, and oversight systems applied to agentic workflows to ensure that agents behave consistently, safely, and within intended boundaries. Effective guardrails include: maximum iteration limits to prevent infinite loops; output validation checks that verify agent outputs before they are acted upon; tool permission scopes that restrict which tools an agent may invoke; human-in-the-loop checkpoints that require human approval before consequential actions are taken; and monitoring and logging systems that provide full observability into agent execution.

1.10 Orchestrating LLMs — Open-Source vs. Proprietary

Understanding the model landscape and provider switching strategies

A production-grade Agentic AI system will typically leverage multiple LLMs from various providers, selected on the basis of cost, capability, latency, and availability. It is therefore essential to understand the two major categories of available models and how to programmatically switch between them.

Open-Source Models

Open-source LLMs are models whose architecture, weights, and training code are publicly released under permissive licenses. This means they can be freely used, modified, fine-tuned, and redistributed. They may be hosted on cloud inference platforms (such as Groq, Together.ai, or Hugging Face Inference API) or run locally on personal hardware.

Prominent examples include:

- **LLaMA (Meta AI)** — LLaMA 2 and LLaMA 3 series: highly capable general-purpose models available in multiple parameter sizes.
- **Mistral / Mixtral (Mistral AI)** — Efficient models with strong performance-to-size ratios; Mixtral is a Mixture-of-Experts architecture.
- **Falcon (TII)** — Open-weight models from the Technology Innovation Institute.
- **Bloom (BigScience)** — A large multilingual model trained collaboratively across many institutions.

Proprietary API Models

Proprietary models are hosted exclusively by their developing organizations and are accessible only via paid API. They generally represent the current state of the art in terms of instruction-following capability, reasoning, and multimodal understanding. The key proprietary providers relevant to this

course are:

- **OpenAI** — GPT-4o, GPT-4o-mini: powerful general-purpose models; the de facto standard for application development.
- **Anthropic** — Claude-3.5-Sonnet, Claude-3-Opus: models with strong safety characteristics and large context windows.
- **Google** — Gemini 1.5 Pro/Flash: multimodal models with very large context windows (up to 1M tokens).

Provider Switching Strategy

Because many providers expose an OpenAI-compatible API, switching providers requires only changing the api_key and base_url parameters when instantiating the client. No application logic needs to change. This architectural decision provides maximum flexibility and allows cost-performance optimization by routing different tasks to different models.

1.11 AI Agent Frameworks Overview

The role of frameworks in abstracting agentic complexity

Building agentic systems from first principles — managing LLM calls, tool integrations, state management, error handling, and agent coordination manually — is an extremely complex engineering undertaking. AI Agent Frameworks exist to abstract away this complexity, providing developers with pre-built components, design patterns, and orchestration primitives that accelerate development and enforce best practices.

These frameworks act as the 'glue' that connects disparate AI components — LLMs, tools, data sources, memory systems, and external services — into coherent, functional agentic applications. They vary significantly in their abstraction level, design philosophy, target use cases, and the degree of control they expose to the developer.

Framework	Type	Key Strength	Abstraction Level
MCP (Model Context Protocol)	Standard / Protocol	Standardized tool connectivity	Foundational
CrewAI	Multi-Agent Framework	Role-based team orchestration	High
LangGraph	Orchestration Framework	Stateful graph-based workflows	Low-Medium

AutoGen (Microsoft)	Multi-Agent Framework	Human-in-the-loop, modular agents	Medium

1.12 Model Context Protocol (MCP)

Anthropic's open standard for LLM-to-tool connectivity

The Model Context Protocol (MCP) is an open-source standard created by Anthropic that defines a **standardized, provider-agnostic interface** through which Large Language Models interact with external tools, systems, and data sources. It is important to understand that MCP is not itself an agentic AI framework — it does not provide agent coordination, memory management, or planning capabilities. Rather, it is a connectivity protocol: a standardized 'language' that enables any compliant LLM to communicate with any compliant tool or data source, regardless of vendor.

MCP occupies the foundational layer at the bottom of the agentic AI stack. It operates by connecting models to external resources via standardized API calls, without prescribing any particular workflow or agent architecture. Its open-source nature and vendor-neutral design make it a community-wide standard rather than a proprietary solution.

Analogy: USB Standard

MCP is to AI tools what USB is to computer peripherals. Just as USB provides a standardized interface that allows any USB device to connect to any USB-compatible computer, MCP provides a standardized interface that allows any MCP-compliant tool to connect to any MCP-compatible LLM — regardless of the underlying hardware or software vendor.

1.13 CrewAI — Multi-Agent Orchestration

A Python-native framework for building collaborative AI agent teams

CrewAI is an open-source, Python-based framework built from the ground up (without LangChain dependencies) specifically for developing and managing multi-agent AI systems. Its central design metaphor is that of a **crew**: a team of specialized AI agents, each with a clearly defined role, a specific goal, and a set of tools — working collaboratively under an overarching mission.

Core Concepts in CrewAI

- **Agent:** An individual AI entity with a defined *role* (e.g., 'Research Analyst'), a *goal* (e.g., 'Gather comprehensive data on the topic'), a *backstory* (contextual persona), and access to specific *tools* (e.g., web search, calculator).

- **Task:** A specific, well-defined unit of work assigned to an Agent, with a clear description, expected output format, and an assigned agent responsible for its execution.
- **Crew:** The collection of Agents and Tasks, along with a process configuration that determines how tasks are executed (sequentially or hierarchically) and how agents collaborate.
- **Guardrails:** CrewAI incorporates guardrails at the task level, validating agent outputs against expected formats and defined quality criteria before passing them downstream.

CrewAI's role-based architecture mirrors effective human team structures, making it highly intuitive for designing complex, multi-faceted AI applications such as automated research pipelines, content creation systems, and multi-step data analysis workflows.

1.14 LangGraph & AutoGen — Advanced Orchestration

Graph-based state machines and modular multi-agent systems

LangGraph

LangGraph is an open-source framework — an extension of the LangChain ecosystem — designed to construct, manage, and scale **complex, stateful agent workflows** using graph-based computational patterns. Unlike higher-level frameworks that abstract workflow logic behind configuration files, LangGraph operates as a low-level orchestration layer that gives developers granular control over agent execution.

LangGraph models an agentic workflow as a **directed graph**, where each *node* represents an operation (an LLM call, a tool invocation, a conditional branch, or a human-in-the-loop checkpoint), and each *edge* represents the flow of execution between nodes. A *shared state object* is passed through this graph, accumulating and modifying context as execution proceeds. Conditional edges enable dynamic routing — the path of execution changes based on the content of the shared state — making LangGraph exceptionally powerful for complex, non-linear workflows.

AutoGen (Microsoft Research)

AutoGen is an open-source multi-agent framework developed by Microsoft Research in collaboration with academic partners including Penn State University and the University of Washington. Its primary design emphasis is on **modularity** and **human-in-the-loop oversight**: AutoGen makes it straightforward to construct systems where multiple AI agents converse with each other — and, crucially, with human participants — to solve complex tasks.

AutoGen is particularly well-suited to scenarios involving code generation and execution, where a human (or a proxy thereof) can review, modify, and approve generated code before it is run. Its conversational multi-agent pattern supports both fully automated pipelines and semi-automated workflows with human oversight.

Dimension	LangGraph	AutoGen
Primary Use Case	Complex stateful AI orchestration	Modular multi-agent conversations
Execution Pattern	Graph nodes & edges with shared state	Conversational agent interactions
Human-in-the-Loop	Supported at checkpoint nodes	First-class design feature
Abstraction Level	Low — high developer control	Medium — modular & composable
Origin	LangChain ecosystem	Microsoft Research
Best For	Non-linear, stateful pipelines	Code generation, collaborative tasks

1.15 Resources, Tools & the LLM Autonomy Cycle

How to augment LLM capability through external resources and tooling

To maximise the effectiveness and autonomy of LLMs within agentic systems, practitioners employ two complementary strategies: providing the model with access to domain-relevant data resources, and equipping the model with functional tools that extend its capabilities beyond pure text generation.

Data Sharing & Context Augmentation

LLMs possess extensive general knowledge from pre-training, but they lack awareness of proprietary, domain-specific, or recent information. Data sharing — providing the model with curated, relevant documents, structured data, or retrieved context as part of the prompt — significantly improves the model's responses on specialized topics. Techniques such as Retrieval-Augmented Generation (RAG) automate this process by dynamically retrieving the most relevant documents from a knowledge base and injecting them into the LLM's context window at inference time.

The LLM Autonomy Cycle

The complete cycle of LLM autonomy — from prompt receipt to goal achievement — can be summarised as a four-stage process: (1) **Prompt Receipt**: the LLM receives an initial task description; (2) **Response Generation**: the LLM reasons and produces an output or action plan; (3) **Tool Execution**: relevant tools are invoked to gather information or perform actions; (4) **Feedback Integration**: tool results are returned to the LLM, updating its context and informing the next generation step. This cycle repeats, with each iteration refining the LLM's understanding and outputs until the task is completed.

SECTION 2

Practical Implementation

This section presents complete, annotated code implementations of the concepts introduced in Section 1. Each IPython notebook cell is presented alongside a comprehensive explanation of its purpose, the Python constructs it employs, and the expected output it produces. All code is designed for execution within a Python virtual environment as configured in Week 1.

Prerequisites

Before executing any of the following code, ensure that: (1) a Python virtual environment has been created and activated; (2) required packages have been installed via pip; (3) a .env file containing valid API keys has been created at the project root; and (4) the .env file has been added to .gitignore.

2.1 Environment Configuration & Package Installation

Setting up the Python environment and installing required libraries

Cell 1 — Installing Required Packages

In [1]:

```
# Install required packages into the active virtual environment
# The exclamation mark (!) runs shell commands from within a Jupyter notebook
!pip install openai python-dotenv
```

Explanation:

This cell installs the two foundational packages required for all subsequent LLM interactions. The `!` prefix is a Jupyter notebook convention that passes the remainder of the line to the underlying operating system shell for execution.

- **openai**: The official OpenAI Python client library. Despite its name, this library's `OpenAI` client class is compatible with multiple providers (Groq, Gemini via OpenAI-compatible endpoints, Anthropic via compatible wrappers) by changing the `base_url` parameter.
- **python-dotenv**: A library that reads key-value pairs from a `.env` file and loads them into the operating system's environment variables, making them accessible to Python code via `os.getenv()`.

Expected Output:

Out [1]:

```
Collecting openai
Downloading openai-1.x.x-py3-none-any.whl (xxx kB)
Collecting python-dotenv
Downloading python_dotenv-1.x.x-py3-none-any.whl (xx kB)
...
Successfully installed openai-1.x.x python-dotenv-1.x.x
```

The output confirms that both packages and their dependencies have been successfully downloaded from PyPI and installed into the active virtual environment. The exact version numbers will depend on the latest available releases at the time of installation.

2.2 Loading Environment Variables Securely

Reading API keys from .env files using dotenv and os modules

Cell 2 — Creating the .env File Structure

Before writing Python code, create a file named `.env` at the root of your project directory with the following structure:

In [.env]:

```
# .env file — store this at the project root, NEVER commit to git
OPENAI_API_KEY=sk-xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
GROQ_API_KEY=gsk_xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
GOOGLE_API_KEY=AIzaSyxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
```

Cell 3 — Loading Environment Variables in Python

In [3]:

```
import os
from dotenv import load_dotenv

# Load variables from .env file into the OS environment namespace
# override=True ensures .env values take precedence over pre-existing OS variables
load_dotenv(override=True)

# Retrieve the API key from the environment
openai_api_key = os.getenv('OPENAI_API_KEY')

# Verify the key was loaded (print only first/last 4 chars for security)
if openai_api_key:
```

```
print(f'API key loaded successfully: {openai_api_key[:4]}...{openai_api_key[-4:]}')
else:
print('ERROR: OPENAI_API_KEY not found. Check your .env file.')
```

Explanation — Line by Line:

- **import os**: Imports Python's built-in operating system interface module. The `os.getenv()` function reads variables from the OS environment namespace — the same namespace that `load_dotenv()` populates.
- **from dotenv import load_dotenv**: Imports the `load_dotenv()` function from the `python-dotenv` package. This function parses the `.env` file and injects each key-value pair into the OS environment.
- **load_dotenv(override=True)**: Reads the `.env` file from the current working directory (or its parents) and loads all key-value pairs. The `override=True` argument forces `.env` file values to replace any existing OS-level environment variables with the same names, ensuring a consistent configuration state.
- **os.getenv('OPENAI_API_KEY')**: Retrieves the value of the `OPENAI_API_KEY` variable from the OS environment namespace. Returns `None` if the variable is not found, avoiding a `KeyError` exception (unlike `os.environ['KEY']`).
- **Validation check**: The conditional block verifies the key was successfully retrieved and prints a partially masked version of the key for confirmation — never print the full API key in a notebook that may be shared.

Expected Output:

```
Out [3]:
API key loaded successfully: sk-p...xxxx
```

This output confirms that the `.env` file was found, parsed correctly, and the API key was successfully injected into the Python runtime environment. If the error message appears instead, verify that the `.env` file exists at the project root and that the variable name is spelled exactly as expected.

2.3 Making Your First LLM API Call

Constructing messages, instantiating the client, and parsing the response

Cell 4 — Complete First API Call

```
In [4]:
from openai import OpenAI

# Instantiate the OpenAI client using the key loaded from the environment
```

```
client = OpenAI(api_key=openai_api_key)

# Construct the messages list — the conversation context
messages = [
{"role": "system", "content": "You are a helpful assistant specialising in AI."},
{"role": "user", "content": "What is 2 + 2? Explain your answer briefly."}
]

# Dispatch the request to the Chat Completions endpoint
response = client.chat.completions.create(
model="gpt-4o-mini", # Model identifier
messages=messages, # The conversation context
max_tokens=200, # Limit response length
temperature=0.7, # Controls randomness (0=deterministic, 1=creative)
)

# Extract and print the generated text from the response object
print(response.choices[0].message.content)
```

Explanation — Construct by Construct:

- **from openai import OpenAI**: Imports the `OpenAI` class — the primary client for interacting with the API. This class encapsulates all authentication, HTTP communication, and response parsing.
- **client = OpenAI(api_key=openai_api_key)**: Creates an authenticated client instance. The `api_key` is used to authenticate all requests made through this client. Note: if the environment variable `OPENAI_API_KEY` is already set, the `OpenAI()` constructor will detect it automatically, making explicit passing optional.
- **messages list**: Constructs the conversational context as a list of role-content dictionaries. The `system` message establishes the AI's persona and behavioural instructions. The `user` message contains the actual query.
- **client.chat.completions.create()**: The core API call. `model` specifies which LLM to use. `max_tokens` caps the length of the generated response. `temperature` controls output randomness — lower values yield more deterministic, focused responses; higher values introduce more variability and creativity.
- **response.choices[0].message.content**: Navigates the response object hierarchy. `choices` is a list accommodating multiple candidate responses (if `n > 1` was specified). `[0]` accesses the first (and typically only) candidate. `.message.content` extracts the raw text string from the message object.

Expected Output:

```
Out [4]:
```

```
2 + 2 equals 4. This is a basic arithmetic addition. When you add
the integer 2 to itself, the result is the integer 4. This holds
true in standard mathematics and in virtually all programming
language implementations of integer arithmetic.
```

The model returns a coherent, well-formed response. The exact wording will vary between executions due to the non-zero temperature setting, but the factual content will be consistent. This demonstrates that the API connection is correctly established and functional.

2.4 Switching Between Multiple LLM Providers

Leveraging the OpenAI-compatible interface for provider flexibility

Cell 5 — Using Groq (Free Tier) as a Drop-in Replacement

In [5]:

```
# Load Groq API key
groq_api_key = os.getenv('GROQ_API_KEY')

# Instantiate the OpenAI client, pointed at Groq's inference endpoint
# Groq exposes an OpenAI-compatible API — only the base_url and api_key change
groq_client = OpenAI(
api_key=groq_api_key,
base_url="https://api.groq.com/openai/v1"
)

# Identical messages list — no changes required
messages = [
{"role": "user", "content": "Explain what an AI agent is in two sentences."}
]

# Call using a Groq-hosted open-source model
response = groq_client.chat.completions.create(
model="llama3-70b-8192", # LLaMA 3 70B hosted on Groq
messages=messages,
)

print(response.choices[0].message.content)
```

Explanation:

This cell demonstrates the power of the OpenAI-compatible API standard. By supplying Groq's `base_url` and a valid Groq API key, the identical `OpenAI` client class seamlessly redirects all requests to Groq's ultra-fast inference infrastructure. The application logic — message construction, response parsing, and downstream processing — remains completely unchanged.

Groq is particularly noteworthy for its free tier, which provides access to powerful open-source models (LLaMA 3, Mixtral, Gemma) with very high throughput. This makes it an excellent choice for learning, prototyping, and cost-conscious production deployments.

Expected Output:

Out [5]:

```
An AI agent is an autonomous system powered by a large language model
that can perceive its environment, make decisions, and take actions to
achieve specific goals. Unlike a simple chatbot, an AI agent can use
external tools, maintain context over multiple steps, and adapt its
behaviour based on feedback from its environment.
```

2.5 Building a Basic Prompt-Response Loop

Maintaining conversational state across multiple turns

Cell 6 — Multi-Turn Conversation Manager

In [6]:

```
def chat(client, model, system_prompt, conversation_history, user_message):
"""
Manages a multi-turn conversation with an LLM.

Args:
client: An instantiated OpenAI-compatible client.
model: Model identifier string.
system_prompt: The system-level instruction string.
conversation_history: A list of prior message dicts (mutated in place).
user_message: The new user input string.

Returns:
The assistant's response as a string.
"""
# Append the new user message to the running history
```

```
conversation_history.append({'role': 'user', 'content': user_message})

# Prepend the system prompt for each API call
full_messages = [{'role': 'system', 'content': system_prompt}] + conversation_history

# Make the API call with the full conversation context
response = client.chat.completions.create(
model=model,
messages=full_messages
)

# Extract the response text
assistant_reply = response.choices[0].message.content

# Append the assistant's reply to history for future turns
conversation_history.append({'role': 'assistant', 'content': assistant_reply})

return assistant_reply

# ■■ Usage Example ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
history = []
system = 'You are a concise AI tutor specialising in machine learning.'

reply1 = chat(client, 'gpt-4o-mini', system, history, 'What is a neural network?')
print('Turn 1:', reply1)
print()

reply2 = chat(client, 'gpt-4o-mini', system, history, 'How does backpropagation work
in that context?')
print('Turn 2:', reply2)
```

Explanation:

- **conversation_history list**: A mutable list that accumulates all prior exchanges. Because the OpenAI API is *stateless* — it has no memory between calls — the full conversation history must be transmitted with every request. This is the standard pattern for implementing conversational memory at the application level.
- **full_messages construction**: The system prompt is prepended at every call. This ensures the model always has its behavioural instructions regardless of how long the conversation grows.
- **Appending both user and assistant messages**: After receiving the model's response, both the user's message and the assistant's reply are stored in `conversation_history`. This enables the model to reference prior turns and maintain coherent multi-turn context.

- **Turn 2 — contextual reference**: The phrase 'in that context' in Turn 2's question is only meaningful because the prior exchange is included in the messages. Without history, the model would not know what context to refer to.

Expected Output:

Out [6]:

```
Turn 1: A neural network is a computational model inspired by the
structure of the human brain. It consists of layers of interconnected
nodes (neurons) that process and transform input data to produce
predictions or classifications.

Turn 2: In the context of neural networks, backpropagation is the
algorithm used to train the network. It computes the gradient of the
loss function with respect to each weight by applying the chain rule
of calculus backwards through the network layers...
```

2.6 Implementing a Simple Prompt Chain

Sequentially refining output through multiple LLM calls

Cell 7 — Two-Stage Prompt Chain: Draft & Refine

In [7]:

```
def prompt_chain(client, model, topic):
"""
A two-stage prompt chain: first drafts an explanation,
then critically refines it for clarity and accuracy.
"""
# ■■ STAGE 1: Generate initial draft ■■■■■■■■■■■■■■■■■■■■■■
stage1_messages = [
{'role': 'system', 'content': 'You are a knowledgeable AI educator.'},
{'role': 'user', 'content': f'Write a brief explanation of: {topic}'}
]
stage1_response = client.chat.completions.create(
model=model, messages=stage1_messages, max_tokens=300
)
draft = stage1_response.choices[0].message.content
print('--- STAGE 1: INITIAL DRAFT ---')
```

```
print(draft)
print()

# ■■ STAGE 2: Critique and refine ■■■■■■■■■■■■■■■■■■■■■■■■
stage2_messages = [
{'role': 'system', 'content': 'You are an expert editor for technical AI content.'},
{'role': 'user', 'content': (
f'Refine the following explanation to be clearer, more precise, '
f'and better suited for an advanced learner. Return only the refined version.\n\n'
f'DRAFT:\n{draft}'
)}
]
stage2_response = client.chat.completions.create(
model=model, messages=stage2_messages, max_tokens=300
)
refined = stage2_response.choices[0].message.content
print('--- STAGE 2: REFINED OUTPUT ---')
print(refined)
return draft, refined

# Execute the chain
draft, refined = prompt_chain(client, 'gpt-4o-mini', 'the attention mechanism in
transformers')
```

Explanation:

This cell implements a functional two-stage Prompt Chain. It demonstrates the core principle of chaining: the output of Stage 1 (the draft) becomes the primary input to Stage 2 (the refiner). This is achieved by embedding `draft` directly into the Stage 2 user message via an f-string.

- **Stage 1 — Generation**: A generalist 'educator' persona is instructed to produce an initial explanation. The output at this stage is comprehensive but may be imprecise or inconsistently structured.
- **Stage 2 — Refinement**: A specialist 'editor' persona receives the draft and is explicitly instructed to improve clarity, precision, and audience-appropriateness. Crucially, it is instructed to return *only* the refined version, preventing preamble or meta-commentary in the output.
- **Return value**: Both the draft and the refined output are returned, enabling downstream comparison, logging, or further processing stages.

Expected Output:

Out [7]:

```
--- STAGE 1: INITIAL DRAFT ---
The attention mechanism allows neural networks to focus on relevant
parts of the input when producing each output token. Instead of
compressing all input information into a fixed vector...

--- STAGE 2: REFINED OUTPUT ---
The attention mechanism in transformer architectures enables each
output token to selectively weight all input tokens via learned
Query, Key, and Value projections. The scaled dot-product attention
formula — softmax(QK^T / sqrt(d_k)) * V — computes a weighted sum...
```

Comparing Stage 1 and Stage 2 outputs clearly demonstrates the value of the prompt chain: Stage 2 is noticeably more precise, correctly introduces technical terminology (Query, Key, Value), and includes the mathematical formulation — details that emerged from the refinement pass rather than the initial generation.

Chapter Summary & Key Takeaways

1. A well-configured development environment — comprising VS Code, Python virtual environments, Git, and the dotenv pattern for API key management — is the non-negotiable foundation of professional Agentic AI development.

2. The OpenAI-compatible API interface is a de facto standard that enables seamless switching between providers (OpenAI, Groq, Gemini, Anthropic) by simply changing the base_url and api_key parameters.

3. An Agent is a system in which LLM outputs dynamically determine the sequence of subsequent actions, fundamentally distinguished from simple query-response interactions by autonomy and tool usage.

4. Workflows (deterministic, predefined paths) and Agents (dynamic, LLM-directed paths) represent complementary paradigms with different trade-offs in predictability, flexibility, and appropriate use cases.

5. Workflow design patterns — Prompt Chaining, Routing, Parallelization, Orchestrator-Worker, and Critic-Optimizer — provide powerful, composable building blocks for structured agentic systems.

6. The Agentic Feedback Loop (initiation → reasoning → action → observation → adaptation → repeat) is the operational mechanism underlying all autonomous agentic behaviour.

7. Guardrails — iteration limits, output validation, tool permission scopes, and human oversight — are essential engineering controls for safe and reliable production agentic systems.

8. MCP, CrewAI, LangGraph, and AutoGen represent the major framework tiers of the Agentic AI ecosystem, ranging from foundational connectivity protocols to high-level multi-agent orchestration frameworks.

End of Week 1 — Agentic AI